

Owmee Operations, Integrations, and Launch Readiness

Version: 2026-06-29

1. Purpose

This document turns Owmee's architecture into an operator-ready launch guide. It covers infrastructure, provider setup, queue ownership, observability, quality gates, and critical backlog items.

2. Launch Infrastructure

Layer	Current design	Launch expectation
API	FastAPI on Render	Production env with strict provider validation
Worker	Python worker on Render	Running for AI drafts and background jobs
DB	Postgres	Managed backups, migration guardrails, connection budget
Redis	OTP, rate limits, streams, revocation	Managed Redis or equivalent with persistence policy
Object storage	Cloudflare R2	Private originals, safe public display URLs
Mobile	React Native Android/iOS	Release builds signed and tested on real devices
Admin	Vite React plus server-rendered admin	Protected admin auth, RBAC, audit log
Monitoring	Sentry/logs	Error alerts, stuck workflow queues, provider metrics

3. Environment And Provider Switches

Capability	Required production env
SMS	SMS_PROVIDER=msg91, SMS_API_KEY, SMS_TEMPLATE_ID, SMS_SENDER_ID, DLT fields
Payment	PA_PROVIDER=razorpay, PA_KEY_ID, PA_KEY_SECRET, PA_WEBHOOK_SECRET
AI	AI_PROVIDER=gemini, Gemini credentials/config
Fraud	FRAUD_PROVIDER=bureau, base URL, API key, paths, timeout
KYC	KYC_PARTNER, partner credentials
Geo	GEOCODING_PROVIDER=photo n or approved provider
Push	PUSH_PROVIDER=fcm with FCM credentials, or noop in private testing
Storage	R2_ENDPOINT, R2_ACCESS_KEY, R2_SECRET_KEY, R2_PUBLIC_URL
Security	SECRET_KEY, CORS origins, Sentry DSN, strict validation before public launch

4. Render Deployment Guardrails

Predeploy must:

- 1 Widen Alembic version table if needed.
- 2 Run migrations to head.

- 3 Fail fast on migration errors.
- 4 Keep provider startup validation aligned with environment.

Operational checks:

```
python -m app.db.predeploy
alembic heads
alembic current
```

The latest expected head is 0050_fe_onboarding.

5. MSG91 And DLT Operations

Owmee uses MSG91 only for login OTP.

Required checks:

- MSG91 OTP product, not campaign/marketing flow.
- OTP template contains ##OTP##.
- DLT entity, header/sender, and content template approved.
- PE-TM chain approved.
- Jio/Airtel/Vi propagation checked through delivery reports.
- Render SMS_TEMPLATE_ID points to the OTP template id.
- OTP_WHITELIST removed from production.

Probe command from Render shell:

```
PYTHONPATH=/app python scripts/probe_msg91_otp.py --phone +919876543210 --confirm-send
```

If probe succeeds but handset receives nothing, Owmee reached MSG91. Continue debugging in MSG91/DLT/operator reports using the request id.

6. Razorpay Operations

Owmee currently uses Razorpay through the payment adapter. For in-app checkout, mobile launches Razorpay SDK, while backend verifies client confirmation and webhook events.

Dashboard setup:

- Use test mode for test keys.
- Configure webhook URL:

```
https://<api-host>/v1/payments/webhook/razorpay
```

Enable:

- payment_link.paid
- refund.processed
- refund.failed

Rules:

- Do not use browser callback as webhook.
- Always verify X-Razorpay-Signature.
- Verify order/payment id and amount.
- Underpayment must not capture the transaction.
- Late capture after cancel/timeout must refund/release safely.
- Refund retry must be idempotent.

7. Gemini Operations

AI listing performance is controlled by:

- Async R2 upload.
- Redis Stream worker.
- Fast vision path.
- Full fallback only when needed.
- Prompt version metrics.
- Provider latency and token instrumentation.

Quality controls:

- Deterministic temperature zero where required.
- Background-invariant product facts.
- Prompt injection resistance.
- Private data and stock image detection.
- Generic title ban.
- Responsible MRP extraction.
- Category-family deterministic validation.

Operations should monitor:

- Draft queue wait time.
- Image processing time.
- Gemini latency.
- Token counts and cached token counts.
- Failure reasons.
- Seller edit/correction rate.
- Publish completion rate by category.

8. Google Maps, Geo, And Address Operations

Owmee uses backend reverse geocoding and mobile address capture. Google Maps mobile SDK configuration must match Android/iOS package and signing fingerprints.

Critical checks:

- Android package com.owmee and release SHA-1 are present on the API key.
- Debug SHA-1 is present only when debug app needs maps.
- Maps SDK for Android/iOS is enabled as needed.
- Billing is enabled on the correct Google Cloud project.
- Backend reverse geocode endpoint returns structured address shape.
- App gracefully retries malformed or failed address responses.

9. FE Operations

FE readiness gates:

- Admin-created FE profile.
- Phone normalized and login possible.

- Device binding.
- Identity/background verification status.
- Training status.
- Category certification.
- Active status.
- Shift check-in.

Assignment controls:

- Candidate, suspended, inactive, terminal, wrong-category, or device-rebind FE cannot receive work.
- FE app only shows assigned work.
- FE evidence capture is mandatory for pickup/QC outcomes.
- FE cannot perform finance or warehouse-only actions.

10. Admin Queue Ownership

Queue	Owner	Purpose
Listing moderation	Ops/moderation	Approve/reject non-auto listings
Seller readiness	Ops	Monitor captured orders waiting on seller
Pickup queue	Ops	Assign FE after readiness
Hub queue	Ops	Route FE delivery or courier
Refund queue	Finance/ops	Manage payment reversals
Return queue	Ops/risk	Buyer return decisions and pickup assignment
Risk queue	Risk/admin	Manual risk step-up/block decisions
Direct acquisition queue	Ops	Assign direct pickup and monitor booking state
Price approvals	Supervisor/admin	Approve FE price revisions over threshold
Finance payout queue	Finance	Post seller ledger credits where permitted
Warehouse queue	Warehouse/admin	Receive custody or quarantine mismatch
Stuck workflows	Ops	Resolve states that exceed SLA

11. Observability

Minimum launch telemetry:

- API request errors by route.
- Payment webhook failures and signature rejects.
- Payment pending timeout count.
- Seller readiness timeout/decline count.
- Pickup assignment delay.
- FE pickup fail reasons.
- Delivery delay and failed handover.
- Refund and dispute counts.
- AI draft latency by stage.
- AI prompt failure and correction rates.
- MSG91 delivery failures by provider reason.

- FE device rebind/suspension events.

12. Security And Fraud Controls

Controls already aligned:

- No buyer/seller chat.
- No meetup flow.
- Structured offers only.
- Immutable snapshots for transaction evidence.
- FE device binding and shift check-in.
- FE category certification.
- Admin audit log.
- Provider-neutral risk decisions.
- KYC only where product policy requires it.

Launch hardening:

- Enforce strict provider startup validation in public production.
- Rotate any exposed provider credentials.
- Keep Render env values out of repo.
- Audit admin roles before launch.
- Confirm R2 public/private URL split.
- Confirm Razorpay webhook secret and MSG91 authkey are not reused in dev.

13. Quality Gates Before Shipping

Backend:

```
cd backend
./venv/bin/python -m pytest tests -q
DATABASE_URL=... SYNC_DATABASE_URL=... ./venv/bin/python -m app.db.predeploy
DATABASE_URL=... SYNC_DATABASE_URL=... ./venv/bin/alembic heads
DATABASE_URL=... SYNC_DATABASE_URL=... ./venv/bin/alembic current
```

Admin:

```
cd admin
npm run build
```

Mobile:

```
cd mobile
npm run tsc
npm run lint:design
npm run test:address-gate
cd android && ./gradlew assembleRelease
```

iOS:

```
cd mobile/ios
xcodebuild -workspace owmee.xcworkspace -scheme owmee -configuration Debug -sdk
iphonesimulator -destination 'generic/platform=iOS Simulator' build
```

14. Launch Blocker Backlog

Critical before public launch:

- Confirm production Redis posture for OTP, queues, rate limits, and revocation.
- Finalize Razorpay live mode env and webhook.

- Confirm MSG91 DLT propagation across major operators.
- Confirm FE fraud controls on real devices.
- Confirm R2 signed upload and public display URL behavior in production.
- Confirm seller payout/KYC provider behavior and finance runbook.
- Confirm courier/manual AWB process for pilot transactions.
- Confirm admin roles, audit logs, and incident ownership.
- Confirm iOS and Android release signing and Google Maps key restrictions.

Important but can be phased:

- Courier real-time status integration.
- Rule-based delivery routing after pilot data.
- Warehouse SKU generation UI for direct acquisition.
- FE masked calling/map launch if approved.
- AI correction dataset and evaluation dashboard.
- Design-system migration for existing allow-listed drift.

15. Incident Playbooks

Payment stuck:

- 1 Check transaction status and payment link/payment id.
- 2 Check Razorpay dashboard event and webhook delivery.
- 3 Verify amount/signature/order id.
- 4 If payment failed/cancelled/expired, release inventory.
- 5 If captured after timeout, refund and keep listing safe.

OTP not delivered:

- 1 Run MSG91 probe.
- 2 Check MSG91 request id and delivery report.
- 3 Check DLT template/header/entity mapping.
- 4 Check operator status and phone format.
- 5 If Owmee delivery failed, delete stored OTP and retry after fix.

AI draft stuck:

- 1 Check listing draft status.
- 2 Check Redis Stream worker logs.
- 3 Check R2 object existence and size.
- 4 Check Gemini provider error and retry count.
- 5 Mark failed after retry exhaustion; mobile should recover.

FE suspicious pickup:

- 1 Freeze booking/transaction.
- 2 Review FE device, check-in, geofence, photos, OTP/QR evidence.
- 3 Move to risk/admin queue.
- 4 Reassign or cancel/refund depending on custody state.

Warehouse mismatch:

- 1 Quarantine payable items.
- 2 Keep buyer-facing listing blocked.
- 3 Review FE evidence and seller final acceptance.
- 4 Finance/risk decides seller ledger correction if needed.

16. Ownership Rule

When a flow crosses product, finance, ops, and provider systems, the backend state machine is the source of truth.

Admin screens are control surfaces, not independent truth stores. Manual provider dashboard actions must be reflected back into Owmee through audited admin endpoints.